

LSTM

FOSSO TIOFOUET Jospin Cabrel

Université de Dschang
Faculté des Sciences
Département de Mathématiques et Informatique
www.univ-dschang.org

22 novembre 2025



- 1 Contexte et Problématique
- 2 Des neurones biologiques aux réseaux de neurones
- 3 Limites des ANN sur les séries temporelles
- 4 Réseaux de Neurones Récurrents
- 5 Long Short-Term Memory (LSTM)
- 6 Applications et performances sur séries temporelles



Pourquoi se concentrer sur les LSTM? (1/2)

Contexte du problème :

- Anticiper le **cold start**
- Très peu de données historiques au démarrage
- Besoin de capturer rapidement des motifs complexes et non linéaires

Les LSTM restent le choix le plus pertinent en 2025 car :

- 1 **Spécialistes des dépendances longues**
Inventés en 1997 exactement pour résoudre le vanishing gradient
- 2 **Apprentissage ultra-rapide**
Les portes identifient immédiatement l'information importante, même avec 20–50 points seulement
- 3 **Robustesse native**
Tolèrent très bien le bruit et les valeurs manquantes (cas typique du cold start)



Pourquoi se concentrer sur les LSTM? (2/2)

4. Référence actuelle – même en 2025 Le survey le plus récent (Kottapalli et al., avril 2025) déclare :

« *Transformer-based foundation models have emerged as a dominant paradigm in time series analysis* »

C'est vrai... mais uniquement quand on a des milliers ou millions de points + pré-entraînement massif, ce qui n'était pas la cas jusqu'à très récemment.

5. Interprétabilité précieuse On peut analyser les portes pour comprendre ce que le modèle retient ou oublie.



Justification du Focus LSTM

Avantages Clés

- **Mémoire sélective** : Capture dépendances longues distances
- **Portes apprenables** : Oubli/mémorisation automatique
- **Robustesse** : Gère bruit et valeurs manquantes
- **Non-linéarités** : Modélise relations complexes temporelles

Applications Concrètes

- **Prévision financière** : Marchés boursiers
- **Signaux médicaux** : ECG, EEG
- **Maintenance prédictive** : Capteurs industriels
- **Météo/Climat** : Données environnementales

💡 **Mais pour comprendre les LSTM, commençons par les fondements...**

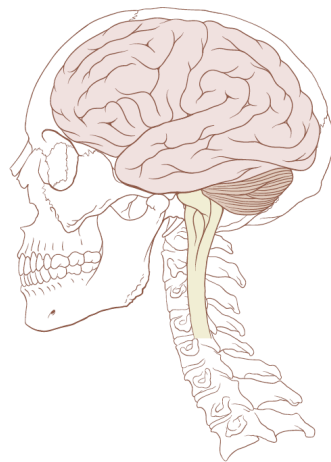


- 1 Contexte et Problématique
- 2 Des neurones biologiques aux réseaux de neurones
- 3 Limites des ANN sur les séries temporelles
- 4 Réseaux de Neurones Récurrents
- 5 Long Short-Term Memory (LSTM)
- 6 Applications et performances sur séries temporelles



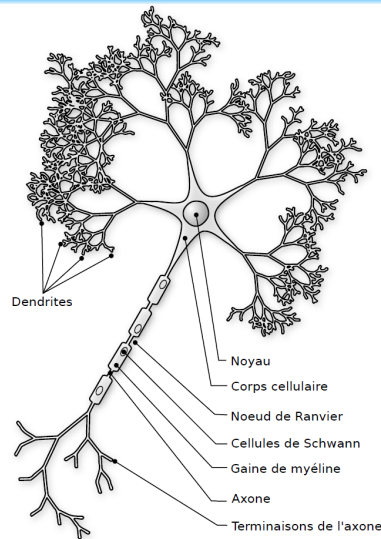
Le cerveau humain en quelques chiffres

-  Nombre de neurones dans le cerveau humain : 100 milliards
- Nombre moyen de connexions par neurone : 10 000
- 1 mm³ de cortex contient 1 milliard de connexions



Le neurone biologique

-  Un neurone est une cellule capable de transmettre des informations à d'autres neurones au travers de ses différentes connexions (synapses)
- Il existe plusieurs types de neurones (pyramide, panier, Purkinje, etc.) avec des fonctionnements différents (sensoriel, moteur, inter-neurones, etc.)
- Les neurone sont inter-connectés et forment des réseaux



Modéliser le cerveau

Pourquoi?

- Pour s'en inspirer
- Pour le comprendre
- Pour le soigner

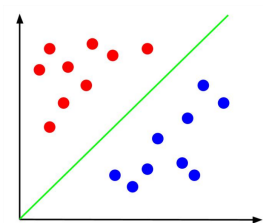
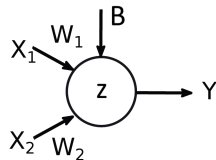
Comment?

- Modéliser un neurone
- Mettre plusieurs neurones en réseau
- Faire apprendre les neurones



Perceptron (1962) → unité de base

- Le perceptron est l'unité de base des réseaux de neurones
- Il s'agit d'un modèle de classification binaire, capable de séparer linéairement deux classes de points



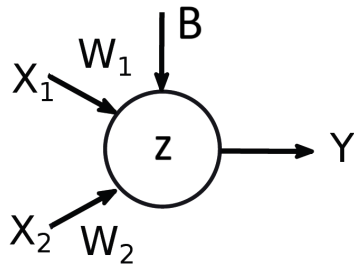
Perceptron (1962) → unité de base

-  Un neurone avec des entrées
- Chaque entrée a un poids
- Une entrée particulière notée B représente le **biais**
- Sortie : fonction des poids et entrées

$$Z = W_1X_1 + W_2X_2 + B$$

$$Y = f(Z)$$

où f est la fonction d'activation

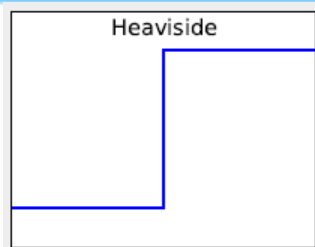


Perceptron simple

Fonctions d'activation et non-linéarité

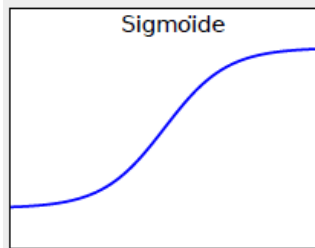
Heaviside (seuil θ)

- Si $x < \theta$ alors $f(x) = 0$
- Si $x \geq \theta$ alors $f(x) = 1$



Sigmoïde / Tangente hyperbolique

- $f(x) = \frac{1}{1+\exp(-x)}$
- $f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$



Perceptron simple

Rôle de l'injection de la non-linéarité

- **? Pourquoi une fonction d'activation non linéaire?**
 - **Classification** : Transforme une combinaison linéaire ($s = w_1 x_1 + w_2 x_2 + b$) en une décision binaire (ex. Heaviside : $y = 1$ si $s \geq 0$, sinon $y = 0$).
 - **Problèmes complexes** : Permet de modéliser des relations non linéaires, essentielles pour des tâches comme XOR dans des réseaux multicouches.
 - **Apprentissage** : Facilite l'optimisation par gradient (ex. sigmoïde, ReLU) pour l'apprentissage par rétropropagation.
 - **Expressivité** : Augmente la capacité du modèle à capturer des patterns complexes dans des réseaux neuronaux.



Perceptron simple

Fonction coût

Comment avoir les meilleures prédictions possibles?

Il suffit de régler les paramètres W et B du perceptron de telle sorte qu'il fasse les plus petites erreurs possibles.

Comment y arriver?

La première étape consiste à définir une fonction coût qui va permettre de mesurer ces erreurs.

$$J = -\frac{1}{m} \sum_i y^{(i)} \cdot \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \cdot \log(1 - \hat{y}^{(i)})$$

- J est la fonction coût,
- m est le nombre d'exemples d'entraînement,
- $\hat{y}^{(i)}$ est la prédiction du modèle pour l'entrée $x^{(i)}$,
- $y^{(i)}$ est la sortie réelle pour l'entrée $x^{(i)}$.



Perceptron simple

Descente du gradient

La deuxième étape consiste à mettre sur pied un algorithme qui soit capable de minimiser cette fonction coût : c'est la **descente du gradient**.

Descente du gradient

Descendre le gradient signifie que l'on cherche à réduire l'erreur dans la direction de l'erreur, en descendant le long du gradient. Si l'on considère les entrées x_i du réseau associés respectivement aux poids w_i , alors :

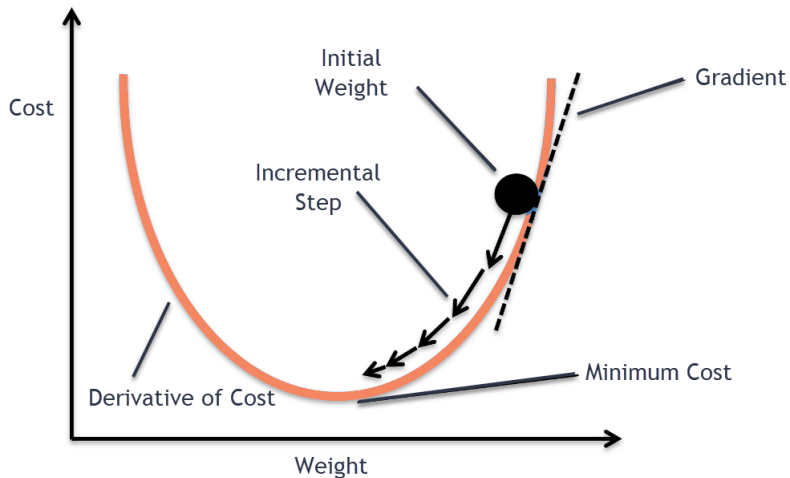
$$w_1^{i+1} \leftarrow w_1^i + \alpha \frac{\partial J}{\partial w_1}$$

α : taux d'apprentissage



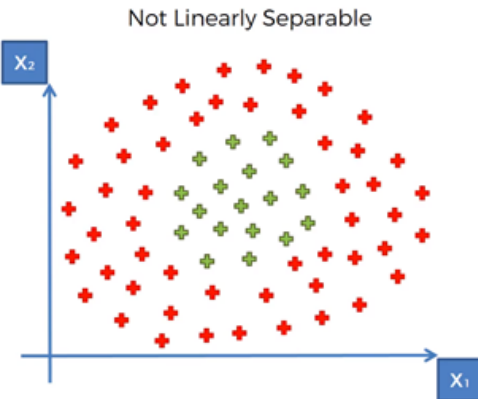
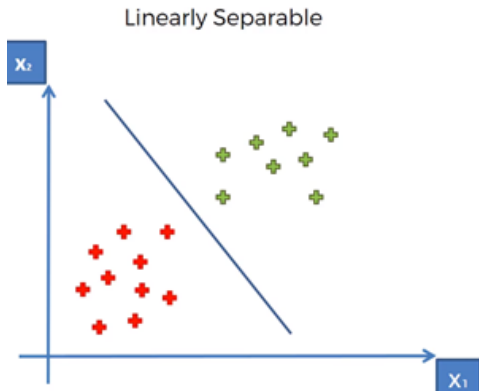
Perceptron simple

Descente du gradient



Perceptron simple

Limites du perceptron simple



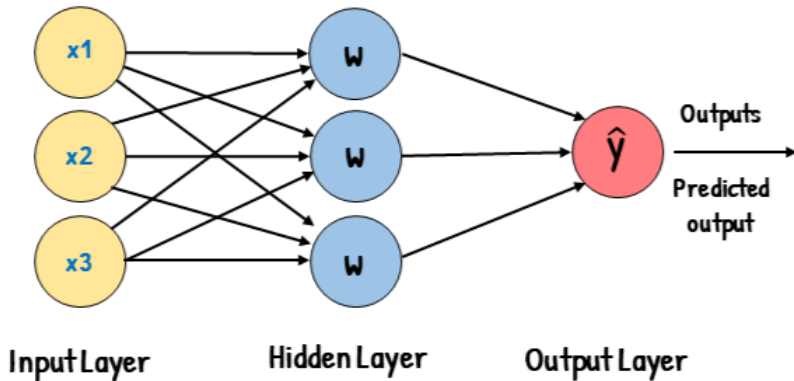
Réseaux de neurones multicouches (MLP ou ANN)

Idée géniale (Rumelhart 1986) : avec plusieurs couches + fonctions d'activation non linéaires → on peut approximer **n'importe quelle fonction** (théorème d'approximation universelle)



Perceptron Multicouches

Réseaux de Neurones Artificiels



Backpropagation vs Descente de Gradient

Descente de Gradient : Algorithme d'optimisation

- But : Minimiser la fonction de coût
- Méthode : $\omega \leftarrow \omega - \alpha \nabla_{\omega} J$

Backpropagation : Algorithme de calcul efficace des gradients

- But : Calculer $\frac{\partial L}{\partial w}$ pour tous les poids w
- Méthode : Application intelligente de la règle de chaîne

Relation entre les deux

Backpropagation **calcule** les gradients → Descente de gradient **utilise** ces gradients pour mettre à jour les poids

La backpropagation rend l'entraînement des réseaux profonds réalisable!



- 1 Contexte et Problématique
- 2 Des neurones biologiques aux réseaux de neurones
- 3 Limites des ANN sur les séries temporelles
- 4 Réseaux de Neurones Récurrents
- 5 Long Short-Term Memory (LSTM)
- 6 Applications et performances sur séries temporelles



Pourquoi les ANN Standards Échouent sur les Données Séquentielles?

⚠ Problèmes Fondamentaux

- **Indépendance des entrées** : Chaque input traité séparément
- **Pas de mémoire** : Impossible de retenir le contexte temporel
- **Taille fixe** : Entrées de longueur fixe requise
- **Pas d'ordre** : La séquence n'est pas préservée

✗ **Inadapté aux séries temporelles**

📈 Conséquences pour les Séries Temporelles

- **Tendances ignorées** : Pas de capture des patterns temporels
- **Saisonnalités perdues** : Cycles saisonniers non modélisés
- **Dépendances temporelles** : Relations temporelles ignorées
- **Performances médiocres** : Résultats insatisfaisants

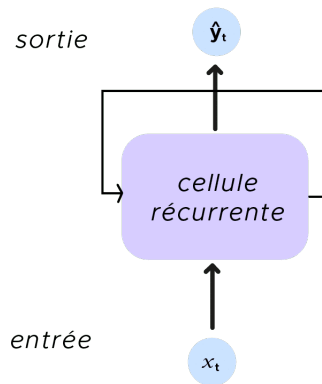


- 1 Contexte et Problématique
- 2 Des neurones biologiques aux réseaux de neurones
- 3 Limites des ANN sur les séries temporelles
- 4 Réseaux de Neurones Récurrents
- 5 Long Short-Term Memory (LSTM)
- 6 Applications et performances sur séries temporelles

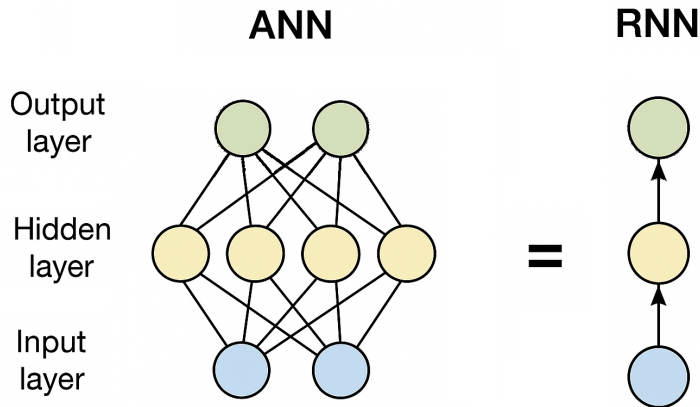


Introduction aux réseaux de neurones récurrents

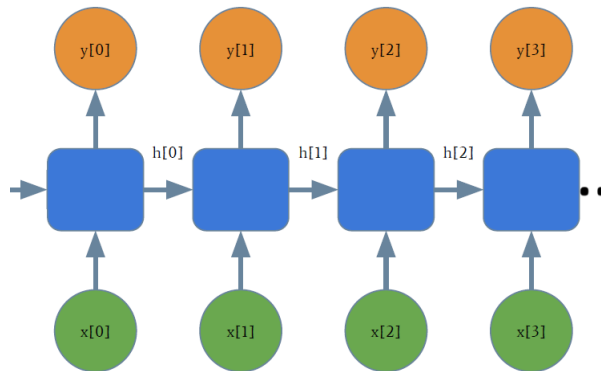
-  **Définition** : Réseaux conçus pour les données séquentielles.
-  **Utilité** : Analyse de texte, séries temporelles, traduction.
-  **Fonctionnement** : Boucle pour conserver une mémoire.



Equivalence ANN - RNN

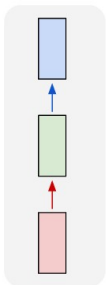


Vue dépliée d'un RNN

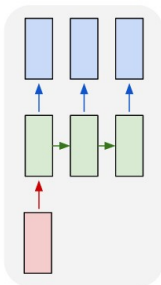


Les différents types de problèmes

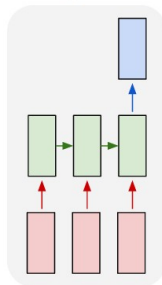
one to one



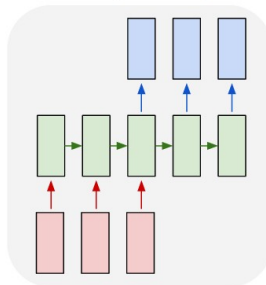
one to many



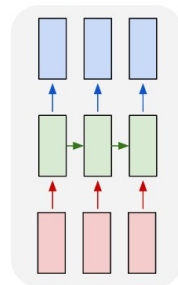
many to one



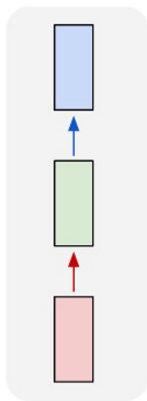
many to many



many to many



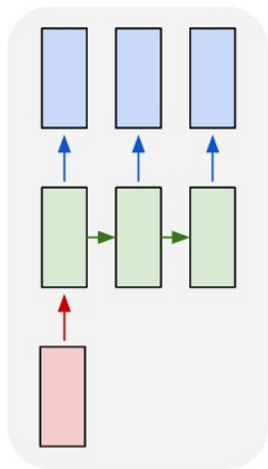
Les différents types de problèmes - One-to-One



- **Entrée** : Un élément
- **Sortie** : Un élément
- **Exemple** :
 - Anti-spam (✉)
 - Attribution d'auteur (👤)
 - Classification d'image (🖼)



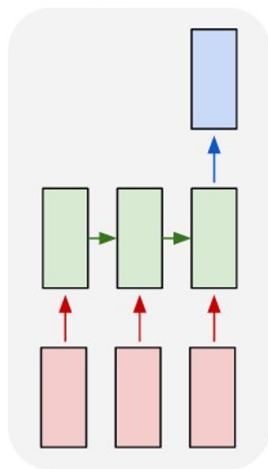
Les différents types de problèmes - One-to-Many



- **Entrée** : Un élément
- **Sortie** : Une séquence d'éléments
- **Exemple** :
 - Génération de la description d'une image (👁️➡️)



Les différents types de problèmes - Many-to-One

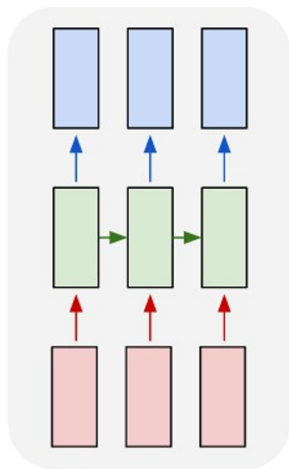


LSTM

- **Entrée** : Une séquence d'éléments
- **Sortie** : Un seul élément
- **Exemple** :
 - Classification du sentiment d'un texte (😊)



Les différents types de problèmes - Many-to-Many



LSTM

- **Entrée** : Une séquence d'éléments
- **Sortie** : Une séquence d'éléments
- **Exemple** :
 - Traduction d'un texte (A z)
 - Résumé (📄)
 - Analyse syntaxique (🔗)



Définition du problème

- Nous considérons le problème **many-to-many**
 - One-to-many et many-to-one peuvent être déduits en cachant des entrées/sorties
- Nous avons une séquence de longueur L
- Séquence d'éléments ordonnés en entrée et sortie :
 - $x[1], x[2], \dots, x[L]$
 - $y[1], y[2], \dots, y[L]$
- En général, on ne peut prédire $y[k]$ qu'en utilisant les entrées $x[i]$, $i \leq k$ précédentes
- On utilise un réseau où la valeur de la couche cachée h est réinjectée comme entrée à la couche cachée de l'étape suivante.



Neurones avec récurrence

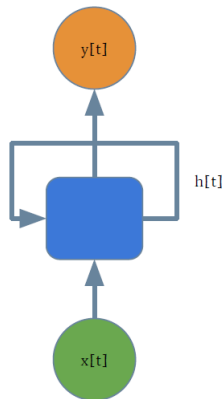
Idée fondamentale Apprendre à encoder l'historique précédent dans un vecteur de taille fixe h_t

Principe des RNN

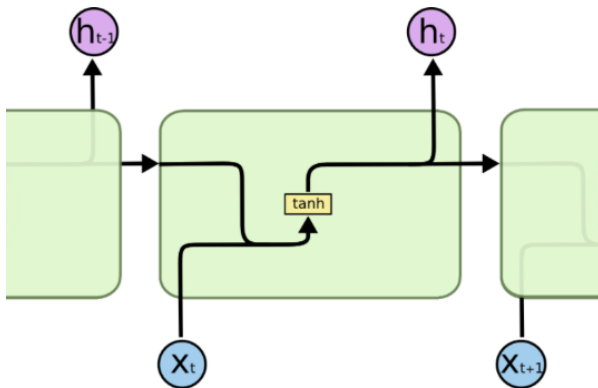
- Séquence d'entrées : $x_1, x_2, \dots, x_n$
- Séquence de sorties : $y_1, y_2, \dots, y_n$
- Dépendances temporelles :
 - $y_i = f(x_i, h_{i-1})$
 - $h_i = g(x_i, h_{i-1})$

Mécanisme clé

- État caché h_t encode l'historique
- Récurrence : h_t réinjecté à $t + 1$
- Partage des paramètres temporels



RNN Vanilla



$$h = \sigma(\tanh[x_t, h_{t-1}] + b)$$



Le Problème des Gradients dans les RNN

! Vanishing Gradients

- **Gradients exponentiels** : Produit de nombreuses matrices Jacobiennes

$$\frac{\partial h_t}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial h_{t-2}} \dots \frac{\partial h_1}{\partial h_0}$$

- **Mémoire courte** : Oublie les informations lointaines
- **Instabilité** : Difficulté d'entraînement
- **Limite pratique** : 10 pas de temps maximum

✗ Conséquences pour les Séries Temporelles

- **Dépendances longues perdues** : Tendances saisonnières ignorées
- **Performances limitées** : Modèles sous-optimaux
- **Applications restreintes** : Séquences longues impossibles



- ① Contexte et Problématique
- ② Des neurones biologiques aux réseaux de neurones
- ③ Limites des ANN sur les séries temporelles
- ④ Réseaux de Neurones Récurrents
- ⑤ Long Short-Term Memory (LSTM)
- ⑥ Applications et performances sur séries temporelles



Solution Avancée pour la Mémoire à Long Terme

Innovations des LSTM

- **Système de portes** : Contrôle flux d'information
- **Deux états** : Mémoire court terme + long terme
- **Oubli sélectif** : Apprend quoi oublier/conservé
- **Chemin mémoire** : Gradient préservé sur long terme

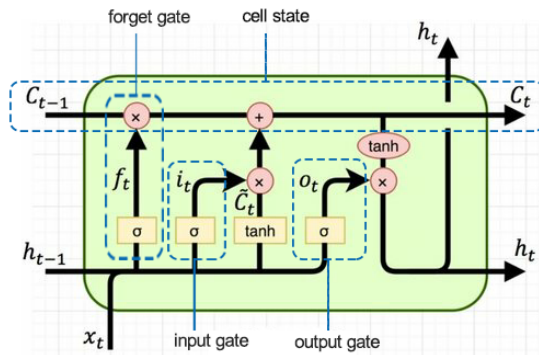
★ Spécialement conçu pour

- Séries temporelles longues
- Dépendances distantes
- Patterns complexes temporels



Long Short-Term Memory (LSTM)

Solution au problème des gradients



- x_t : Input à l'instant t
- h_{t-1} : Mémoire à court terme
- h_t : Nouvelle mémoire à court terme
- C_{t-1} : Mémoire à long terme
- C_t : Nouvelle mémoire à long terme
- h_t : Output à l'instant t



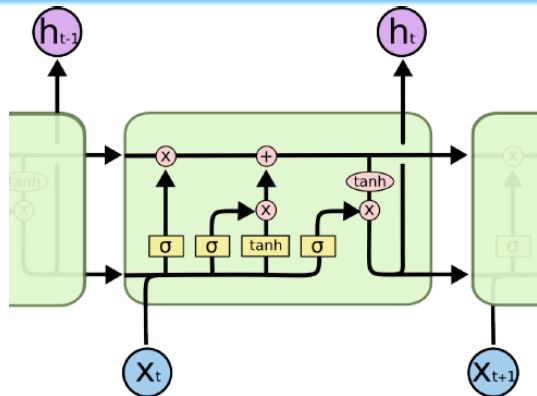
Long Short-Term Memory (LSTM)

Solution au problème des gradients

Idée Fondamentale

- **Deux chemins** d'information :
 - Chemin profond (transformations)
 - **Chemin mémoire** (simple)
- Mécanisme de **portes** contrôlant :
 - Oubli (forget gate)
 - Mise à jour (input gate)
 - Sortie (output gate)

Avantage Clé : Gradient préservé sur le chemin mémoire $\Rightarrow$ Apprentissage des dépendances longues



Long Short-Term Memory (LSTM)

Les portes d'une LSTM

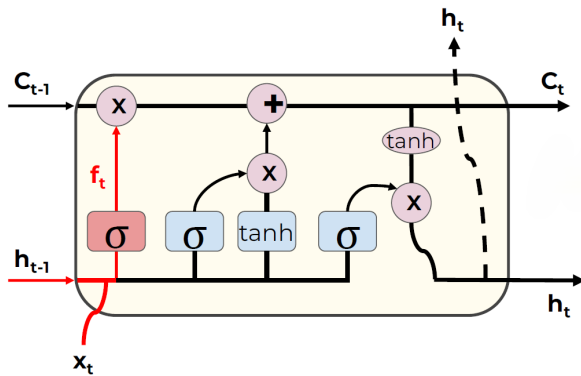
Une cellule LSTM (Long Short-Term Memory) utilise plusieurs portes pour gérer le flux d'information :

- **Porte d'oubli (f_t)** : Détermine quelles informations de l'état précédent doivent être oubliées. Elle permet au réseau de "faire de la place" pour de nouvelles informations pertinentes.
- **Porte d'entrée (i_t)** : Contrôle quelles nouvelles informations seront ajoutées à l'état de la cellule. Elle régule la quantité d'information provenant de l'entrée actuelle et de l'état caché précédent à intégrer.
- **Porte de mise à jour (ou cellule candidate $\tilde{C}_t$)** : Génère les nouvelles valeurs candidates pour l'état de la cellule. Ces valeurs sont ensuite modulées par la porte d'entrée avant d'être ajoutées.
- **Porte de sortie (o_t)** : Décide quelles informations de l'état de la cellule seront sorties vers l'état caché. Elle joue un rôle clé dans le contrôle de l'influence de la mémoire à long terme sur la sortie actuelle.



Long Short-Term Memory (LSTM)

Forget Gate

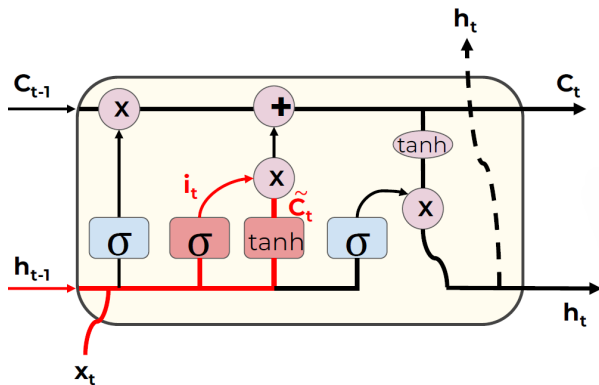


$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$$



Long Short-Term Memory (LSTM)

Input Gate



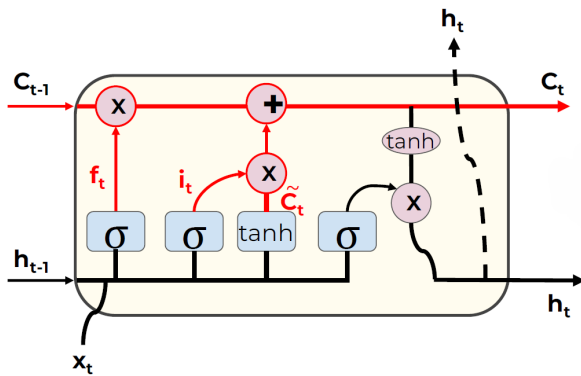
$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i)$$

$$\tilde{c}_t = \tanh(W_C[x_t, h_{t-1}] + b_C)$$



Long Short-Term Memory (LSTM)

Long Memory state

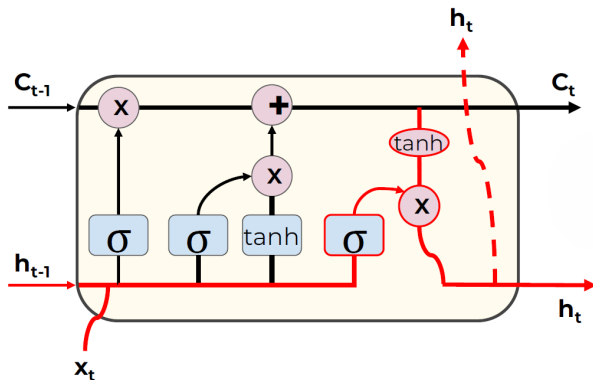


$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$



Long Short-Term Memory (LSTM)

Output Gate



$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$



LSTM : Apprentissage des poids

Les LSTM apprennent les poids de leurs connexions grâce à l'algorithme de **rétropropagation dans le temps** (BPTT) :

- **Propagation avant** : à chaque pas de temps, les valeurs des portes (f_t, i_t, o_t) sont calculées à l'aide de combinaisons linéaires (poids + biais) suivies de fonctions d'activation (sigmoïde ou tanh).
- **Calcul de la perte** : une fonction de coût (souvent l'erreur quadratique ou l'entropie croisée) mesure l'écart entre la sortie produite et la sortie attendue.
- **Rétropropagation dans le temps** : les gradients de la perte sont calculés en remontant le temps, à travers les différentes cellules LSTM, pour chaque poids impliqué dans le calcul des portes.
- **Mise à jour des poids** : les poids des connexions (W_f, W_i, W_o, W_c , etc.) sont ajustés via la descente de gradient (souvent optimisée avec Adam, RMSprop, etc.) pour minimiser la perte.

Ainsi, les poids utilisés dans le calcul des portes sont appris automatiquement pour optimiser les performances du réseau sur une tâche donnée.



- 1 Contexte et Problématique
- 2 Des neurones biologiques aux réseaux de neurones
- 3 Limites des ANN sur les séries temporelles
- 4 Réseaux de Neurones Récurrents
- 5 Long Short-Term Memory (LSTM)
- 6 Applications et performances sur séries temporelles



Pourquoi les LSTM Excellent sur les Séries Temporelles?

Capacités Clés

- **Tendances longues** : Capture cycles saisonniers étendus
- **Apprentissage automatique** : Portes adaptatives aux patterns
- **Robustesse bruit** : Oublie les informations non pertinentes
- **Non-linéarités complexes** : Relations temporelles sophistiquées
- **Mémoire sélective** : Retient seulement l'information utile

Applications Séries Temporelles

- **Prévision financière** : Marchés boursiers, devises
- **Signaux médicaux** : ECG, EEG, monitoring patients
- **Maintenance prédictive** : Capteurs industriels, IoT
- **Météo/Climat** : Données environnementales
- **Énergie** : Prévision consommation électrique



Comparaison LSTM vs RNN Simple vs ANN

ANN Standard

- ✗ Pas de mémoire
- ✗ Taille fixe
- ✗ Séquences ignorées
- Performance : Faible

RNN Simple

- ✓ Mémoire courte
- ✓ Séquences gérées
- ✗ Gradients vanishing
- Performance : Moyenne

LSTM

- ✓ Mémoire longue
- ✓ Séquences optimisées
- ✓ Gradients stables
- Performance : Élevée

🏆 Architecture optimale pour séries temporelles longues



1997 → 2017 → 2025 : même problème, solutions différentes

LSTM (1997) — Première solution robuste

- **Innovation clé** : Portes (**forget**, **input**, **output**) + cellule de mémoire
- **Performance** : Transmet l'information sur **centaines/milliers** de pas de temps

Transformers (2017) — Nouveau paradigme

- **Innovation clé** : Self-attention — chaque position regarde **directement** toutes les positions
- **Avantage** : Traitement **parallèle** de toute la séquence
- **Objectif** : Capturer les dépendances longues, **même ambition** que les LSTM

**Non pas une rupture, mais une nouvelle réponse innovante
à la même question fondamentale**



Perspective : des LSTM aux Transformers

La maîtrise des LSTM ouvre la voie vers les Transformers pour séries temporelles, qui reprennent le même défi des dépendances longues avec une approche différente.

- **LSTM** : Capture séquentielle des dépendances via les portes
- **Transformers** : Capture parallèle via l'attention globale
- **Objectif commun** : Modéliser les relations à long terme

Prochaine étape : explorer cette évolution architecturale



Merci!

♥ Merci de votre attention!

❓ Des questions?

